Banco de Preguntas – Documentación del Sprint 1

Aplicación de escritorio · Java 25 + Swing · JAR Maven

Sprint 1 · Primera iteración

Integrantes

- Martin Silva Cabrera
- Juan Manuel Garcia Diaz
- Yineidy Mamian Jimenez
- Britney Benitez
- Jhonatan

Repositorio: github.com/britneybenitez-gif/banco-preguntas-saber-pro

1. Introducción

Este documento reúne el trabajo de la primera iteración (Sprint 1) del Banco de Preguntas, una aplicación de escritorio en Java 25 + Swing que permite a docentes autores registrar preguntas con validaciones, enviarlas a revisión y consultarlas con paginación y filtros, y a un administrador asignar revisores que reciben una notificación por email (simulado).

Incluye las historias de usuario con sus criterios de aceptación, los prototipos y el test de usabilidad, la planificación del sprint, el escenario de calidad de modificabilidad, la arquitectura con el modelo C4, los patrones de diseño aplicados y los enlaces al video y al repositorio.

2. Backlog del Sprint 1

ID	Historia	Prioridad	Pts	Sprint	Estado
HU-01	Crear pregunta con validaciones	Alta	8	1	Hecha
HU-02	Enviar pregunta a revisión	Alta	5	1	Hecha
HU-03	Listar mis preguntas (paginación y filtros)	Alta	8	1	Hecha
HU-04	Asignar revisores y	Alta	8	1	Hecha (email simulado)

ID	Historia	Prioridad	Pts	Sprint	Estado
	notificar por email				

3. Historias de usuario y criterios de aceptación (implementadas en la iteración 1)

HU-01 · Crear pregunta (Autor)

Como docente autor **quiero** registrar una pregunta con todos sus campos **para** aportarla al banco con calidad estructural.

- **CA1.** Dado el formulario, cuando falta contexto, pregunta directa, un distractor, respuesta, justificación, bibliografía, competencia, tema, subtema o dificultad, entonces se muestra «El campo X es obligatorio» y no se guarda.
- **CA2.** La respuesta correcta solo puede ser A, B, C o D.
- **CA3.** Cada distractor tiene ≥10 caracteres y al menos una palabra de más de 2 letras; no hay distractores repetidos ni iguales a la pregunta.
- **CA4.** No se admite «Todas las anteriores» ni «Ninguna de las anteriores» (sin importar tildes/mayúsculas).
- **CA5.** Al guardar, la pregunta queda en estado *Borrador* asociada al autor.

HU-02 · Enviar a revisión (Autor)

Como autor **quiero** enviar mi borrador a revisión **para** que sea evaluado.

- **CA1.** Solo el autor dueño puede enviarla; otro usuario recibe «No tiene permisos…».
- **CA2.** Solo desde *Borrador*; pasa a *Pendiente de revisión*. Otro estado → error.
- **CA3.** Cada estado se ve con color propio y texto con contraste legible (Borrador gris, Pendiente amarillo, En revisión azul, etc.).
- **CA4.** Una pregunta inexistente muestra «La pregunta no existe».

HU-03 · Listar mis preguntas (Autor)

Como autor **quiero** ver mis preguntas paginadas y filtradas **para** ubicar y dar seguimiento rápido.

- **CA1.** Solo aparecen preguntas del autor en sesión; ningún filtro amplía a otros autores.
- **CA2.** Paginación con anterior/siguiente y total de páginas (página base 1, tamaño ≥1).
- **CA3.** Filtros por estado, dificultad, competencia, tema y texto libre (contexto, enunciado, competencia, tema, subtema), ignorando tildes y mayúsculas.
- **CA4.** Si tras filtrar la página queda fuera de rango, se muestra la última página con resultados.
- **CA5.** Se puede abrir el detalle de una pregunta.

HU-04 · Asignar revisores (Administrador)

Como administrador **quiero** asignar revisores a preguntas pendientes **para** iniciar su evaluación.

- **CA1.** Solo un administrador ve y ejecuta la asignación.
- **CA2.** Solo se listan/aceptan preguntas en *Pendiente de revisión* (de todos los autores).

- **CA3.** Se exige al menos un revisor; deben existir y tener rol Revisor; los repetidos se descartan.
- **CA4.** Al asignar, la pregunta pasa a *En revisión* y cada revisor recibe un email con id, competencia, tema y dificultad.
- **CA5.** Si un email falla, la asignación se conserva y se informa quién no fue notificado.

4. Prototipos de la interfaz de usuario y test de usabilidad

Prototipos

La interfaz tiene tres pantallas Swing, una por rol y flujo principal.

Pantalla	Rol	HU	Elementos clave
CrearPreguntaPanel	Autor	HU-01	Formulario con contexto, pregunta directa, distractores, respuesta correcta (A, B, C o D), justificación, bibliografía, competencia, tema, subtema y dificultad; mensaje «El campo X es obligatorio»
MisPreguntasPanel + PaginadorPanel	Autor	HU-02, HU-03	Tabla de mis preguntas con estado en color, filtros (estado, dificultad, competencia, tema, texto libre), anterior/siguiente con total de páginas, botón Enviar a revisión y detalle
AsignarRevisoresPanel	Administrador	HU-04	Lista de preguntas en Pendiente de revisión, selección de revisores y confirmación con aviso de quién no fue notificado

Los prototipos de las pantallas se describen en la tabla anterior.

Test de evaluación de usabilidad

Objetivo: comprobar que un autor y un administrador completan los flujos principales sin ayuda.

Tarea	Rol	Criterio de éxito	Qué se mide
Crear una pregunta completa	Autor	Queda guardada en Borrador	Tiempo, errores de validación, éxito sí/no
Corregir un campo obligatorio vacío	Autor	Entiende el mensaje y guarda	Éxito sí/no
Enviar un borrador a revisión	Autor	La pregunta pasa a Pendiente de revisión	Tiempo, éxito sí/no
Encontrar una pregunta con filtros y paginación	Autor	Abre el detalle correcto	Tiempo, clics
Asignar dos revisores a una pregunta	Administrador	La pregunta pasa a En revisión	Tiempo, éxito sí/no

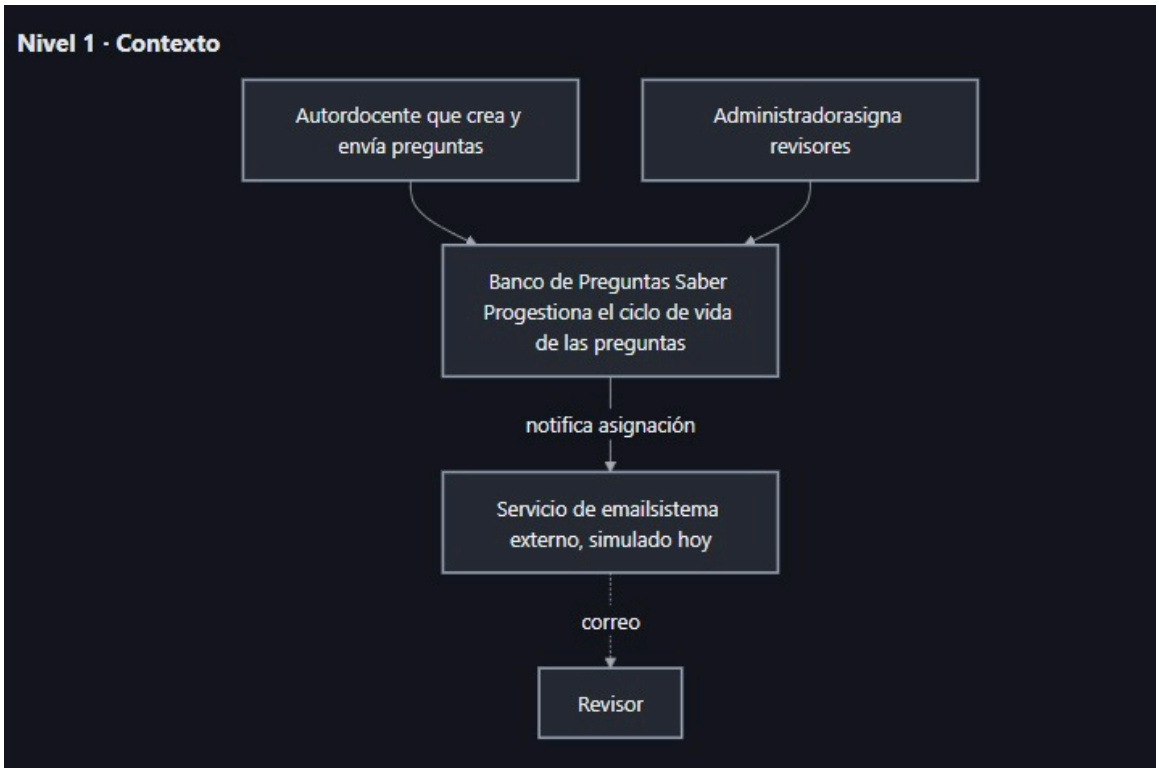
Cuestionario de satisfacción (escala 1 a 5, de totalmente en desacuerdo a totalmente de acuerdo):

1. Los estados con color se distinguen fácilmente.
2. Los mensajes de error me indicaron qué corregir.
3. Encontré mis preguntas rápido con los filtros.
4. Sabía qué hacer en cada pantalla sin ayuda.
5. Volvería a usar la aplicación.

Pendiente: aplicar el test y registrar aquí los resultados.

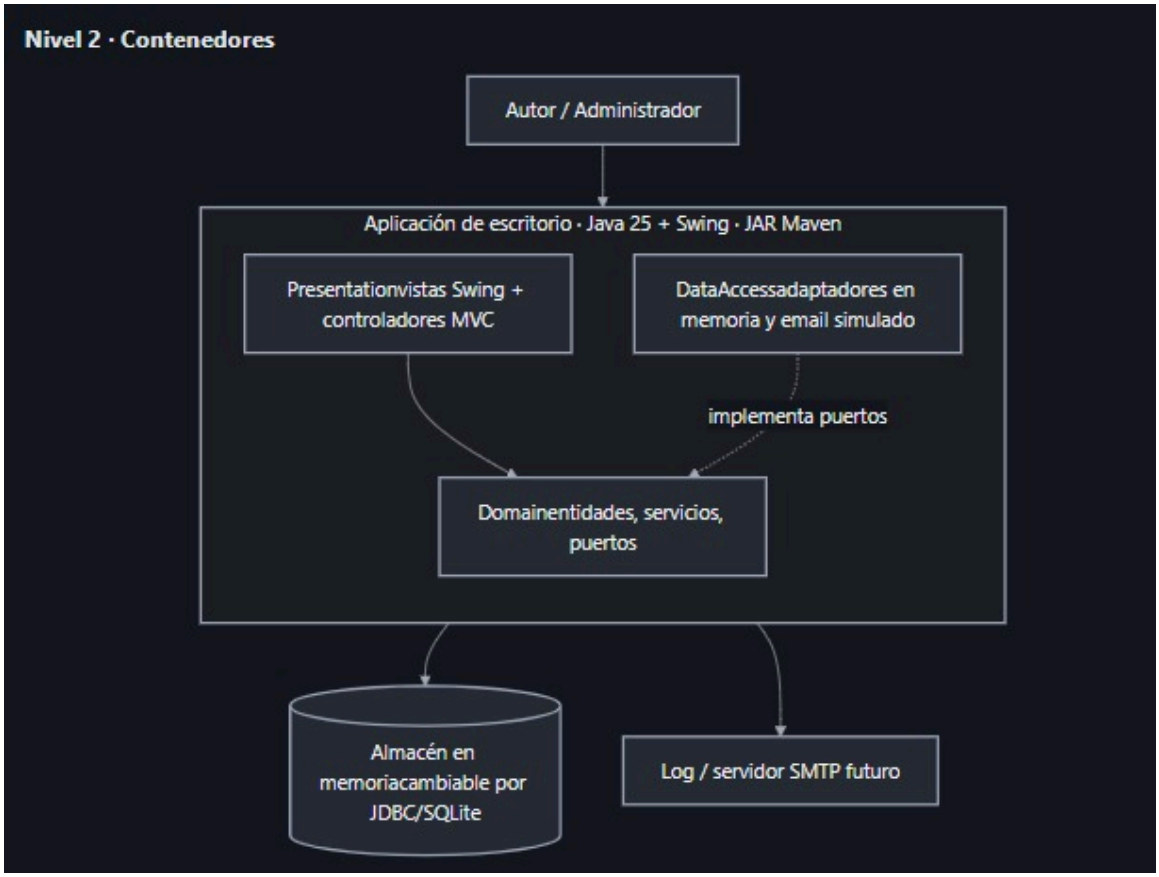
5. Arquitectura (modelo C4)

Nivel 1 · Contexto



El **autor docente** crea y envía preguntas, y el **administrador** asigna revisores, ambos sobre el **Banco de Preguntas Saber Pro**, que gestiona el ciclo de vida de las preguntas. Al asignar, el banco notifica a un **servicio de email** (sistema externo, simulado hoy), que envía el correo al **revisor**.

Nivel 2 · Contenedores



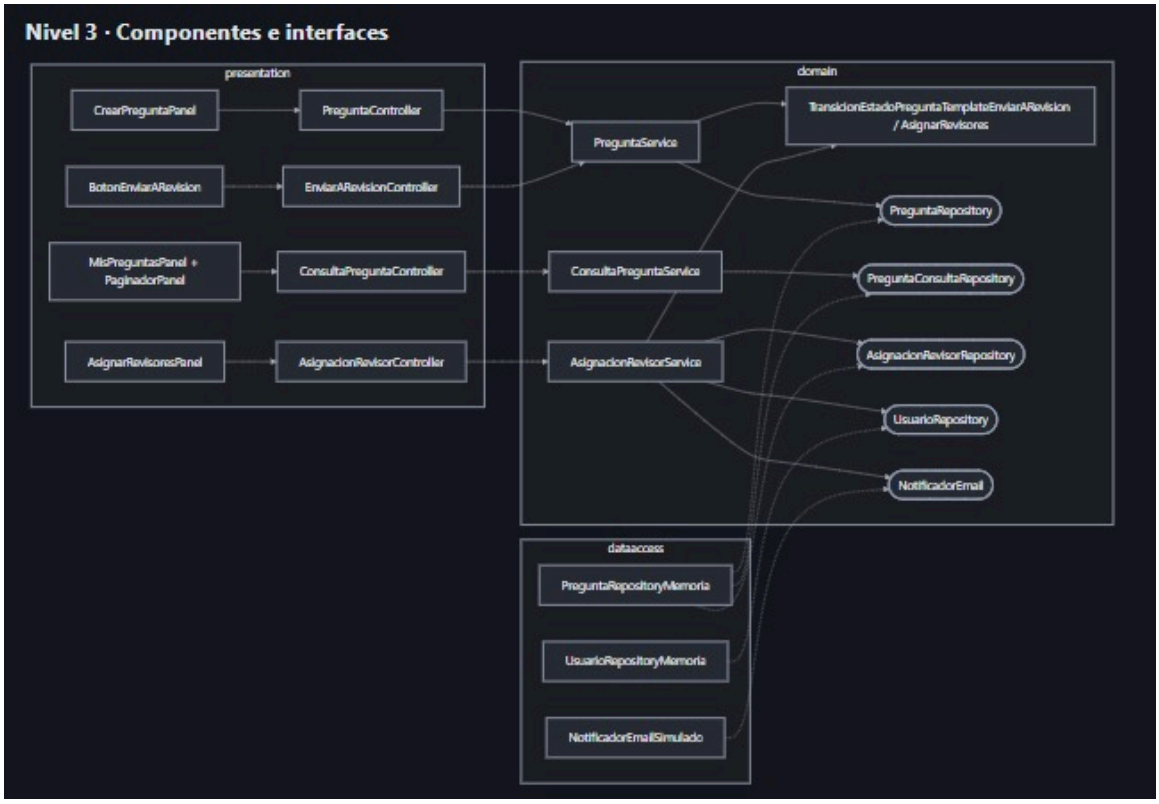
El **Autor / Administrador** interactúa con una **aplicación de escritorio (Java 25 + Swing · JAR Maven)**, compuesta por tres capas:

Capa	Responsabilidad
Presentation	Vistas Swing + controladores MVC
Domain	Entidades, servicios y puertos (interfaces)
DataAccess	Adaptadores en memoria y email simulado; implementa los puertos del dominio

Contenedores externos:

- **Almacén en memoria**, cambiable por JDBC/SQLite.
- **Log / servidor SMTP futuro.**

Nivel 3 · Componentes e interfaces



Paquete `presentation` (cada vista usa su controlador, y este su servicio):

Vista	Controlador	Servicio (domain)
CrearPreguntaPanel	PreguntaController	PreguntaService
BotonEnviarARevision	EnviarARevisionController	PreguntaService
MisPreguntasPanel + PaginadorPanel	ConsultaPreguntaController	ConsultaPreguntaService
AsignarRevisoresPanel	AsignacionRevisorController	AsignacionRevisorService

Paquete `domain`:

- Servicios: `PreguntaService`, `ConsultaPreguntaService`, `AsignacionRevisorService`.
- `TransicionEstadoPreguntaTemplate` (EnviarARevision / AsignarRevisores), usada por `PreguntaService` y `AsignacionRevisorService`.
- Puertos (interfaces): `PreguntaRepository`, `PreguntaConsultaRepository`, `AsignacionRevisorRepository`, `UsuarioRepository`, `NotificadorEmail`.

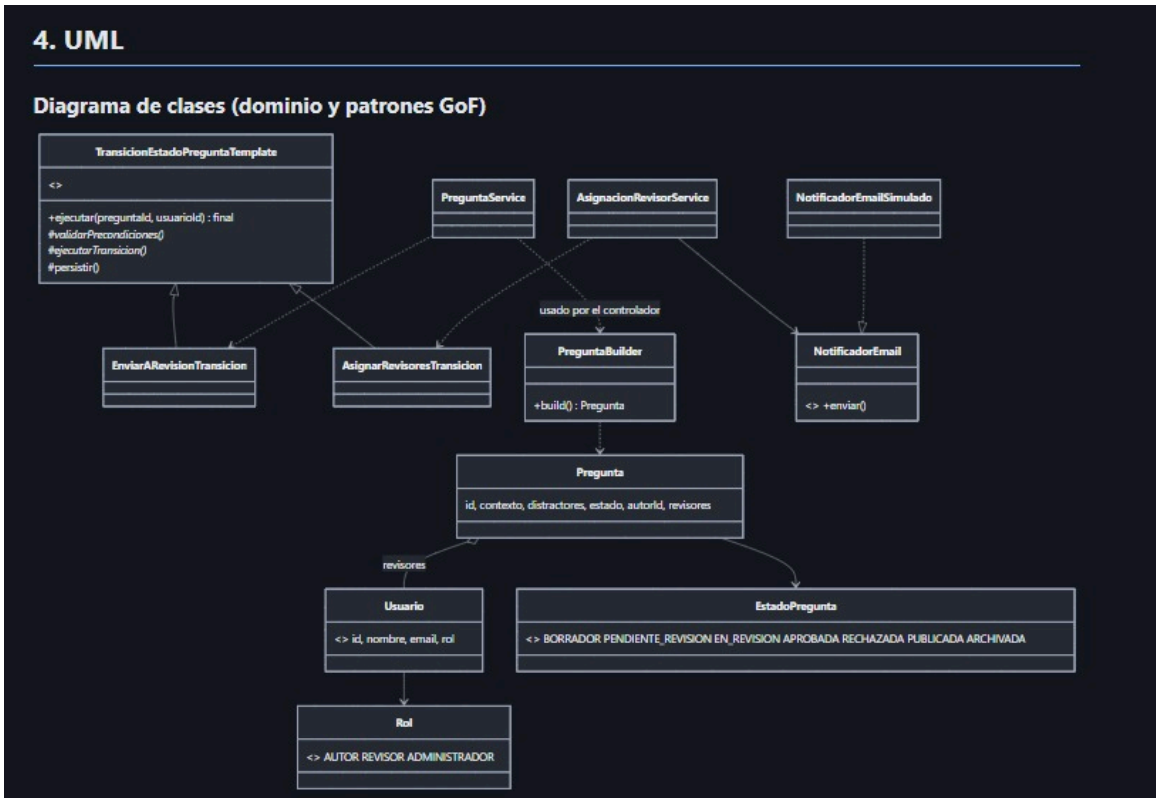
Paquete `dataaccess` (implementan los puertos):

- `PreguntaRepositoryMemoria`
- `UsuarioRepositoryMemoria`

- `NotificadorEmailSimulado`

6. UML

Diagrama de clases (dominio y patrones GoF)



Patrón Template Method

- `TransicionEstadoPreguntaTemplate` (abstracta)
  - `+ejecutar(preguntaId, usuarioId)` : final
  - `#validarPrecondiciones()` : abstracto
  - `#ejecutarTransicion()` : abstracto
  - `#persistir()`
- Subclases: `EnviarARevisionTransicion` y `AsignarRevisoresTransicion`.

Patrón Builder

- `PreguntaBuilder` con `+build() : Pregunta`, usado por el controlador/servicio (`PreguntaService`).

Servicios y puertos

- `PreguntaService` y `AsignacionRevisorService` usan las transiciones; `AsignacionRevisorService` usa `NotificadorEmail`.
- `NotificadorEmail` (interfaz, `+enviar()`) es implementada por `NotificadorEmailSimulado`.

Entidades del dominio

Clase / Enum	Atributos / valores
Pregunta	id, contexto, distractores, estado, autorId, revisores
Usuario	id, nombre, email, rol
EstadoPregunta (enum)	BORRADOR, PENDIENTE_REVISION, EN_REVISION, APROBADA, RECHAZADA, PUBLICADA, ARCHIVADA
Rol (enum)	AUTOR, REVISOR, ADMINISTRADOR

Relaciones: `Pregunta` tiene un `EstadoPregunta` y agrega una lista de `Usuario` como *revisores*; `Usuario` tiene un `Rol`.

7. Atributos de calidad relevantes

Atributo	Cómo se evidencia en el código
Modificabilidad	Repositorios y `NotificadorEmail` son interfaces del dominio; cambiar memoria por BD o email real solo toca `dataaccess` y `App`.
Testabilidad	Inyección por constructor; los servicios se prueban sin Swing (JUnit 5, repositorios en memoria).
Seguridad (control por rol)	Solo el autor dueño envía a revisión; solo el administrador asigna; el listado nunca sale del alcance del autor.
Usabilidad	Estados con color y contraste calculado, paginación, filtros sin tildes, mensajes de validación claros.
Fiabilidad	Si el email falla, la asignación no se revierte y se informa quién quedó sin notificar.
Rendimiento	Consultas paginadas en lugar de cargar todo el banco.

Escenario de calidad: modificabilidad

Elemento	Descripción
Contexto	La aplicación usa repositorios en memoria y un notificador de email simulado, ambos detrás de

Elemento	Descripción
	interfaces del dominio (`PreguntaRepository`, `NotificadorEmail`).
Estímulo	El equipo debe reemplazar el almacén en memoria por una base de datos (JDBC/SQLite) y el email simulado por un servidor SMTP real.
Respuesta	Se crean nuevas implementaciones de los puertos en `dataaccess` y se cambia su inyección en `App`; `domain` y `presentation` no se modifican.
Medición de la calidad	Clases modificadas fuera de `dataaccess` y `App` (meta: 0) y pruebas JUnit 5 de los servicios que siguen pasando sin cambios.
Resultado esperado	El cambio queda localizado en la capa de acceso a datos y no afecta las reglas de negocio ni las vistas.

8. Planificación de tareas del Sprint 1 · Tablero del Sprint 1 (para armar en GitHub Projects / Trello)

Tarea	HU	Columna final
Estructura Maven y paquetes (presentation/domain/dataaccess)	—	Hecho
Entidad Pregunta + PreguntaBuilder	HU-01	Hecho
PreguntaService con validaciones + pruebas	HU-01	Hecho
CrearPreguntaPanel + PreguntaController	HU-01	Hecho
EstadoPregunta, Template Method y EnviarARevisionTransicion	HU-02	Hecho
BotonEnviarARevision + EstadoCellRenderer (colores)	HU-02	Hecho
Filtro, paginación y ConsultaPreguntaService	HU-03	Hecho

Tarea	HU	Columna final
MisPreguntasPanel, PaginadorPanel, detalle	HU-03	Hecho
AsignacionRevisorService, NotificadorEmail (puerto)	HU-04	Hecho
AsignarRevisoresPanel	HU-04	Hecho
Pruebas de consulta y asignación + README de arquitectura	HU-03/04	Hecho

Las tareas del Sprint 1 se listan en la tabla anterior, con su historia de usuario y columna final.

9. Patrones de diseño implementados

Patrón	Clases	Cómo y en qué contexto se aplicó
Template Method (GoF)	`TransicionEstadoPreguntaTemplate`, `EnviarARevisionTransicion`, `AsignarRevisoresTransicion`	HU-02 y HU-04 cambian el estado de una pregunta con los mismos pasos: validar precondiciones, ejecutar la transición y persistir. `ejecutar()` es final y fija ese orden; cada subclase implementa la validación y la transición propias.
Builder (GoF)	`PreguntaBuilder`	HU-01: `Pregunta` tiene muchos campos (contexto, distractores, respuesta, justificación, competencia, tema, etc.). El builder la arma paso a paso y `build()` entrega la pregunta lista para el servicio.
Puertos y adaptadores (Repository)	`PreguntaRepository`, `UsuarioRepository`, `NotificadorEmail` y sus implementaciones en `dataaccess`	El dominio depende de interfaces; la memoria y el email simulado son adaptadores intercambiables. Sustenta la modificabilidad y las pruebas sin Swing.
MVC	Paneles Swing y controladores de `presentation`	Separa las vistas de la lógica: cada panel delega en su

Patrón	Clases	Cómo y en qué contexto se aplicó
		controlador, y este en el servicio del dominio.
Inyección de dependencias por constructor	Servicios y controladores	Los servicios reciben sus repositorios y el notificador, lo que permite probarlos con repositorios en memoria.

Los dos primeros son patrones GoF; los otros tres son patrones arquitectónicos o de diseño generales.

10. URL del video de YouTube

URL: https://youtu.be/m5YG1WERcLg

11. URL del repositorio Git

https://github.com/britneybenitez-gif/banco-preguntas-saber-pro.git